

DOSSIER AGORA MISSION 5.1 À 5.4

URL : 172.17.3.213:8000

Login : leo

Password : leo

Table des matières

1. Contexte	2
1.2 Composant Doctrine	3
1.3 Entités	4
1.4 Associations many to ones et many to many.....	5
a) Associations many-to-one / one-to-many	5
b) Associations many-to-many	6
2. Présentation de l'équipe et organisation du groupe	7
3. Organisation du travail	8
3.1 Trello	8
3.2 Github	9
3.3 Outils utilisés	10
4. Développement et processus du Sprint	11
4.1 Mission 1 : Création de la BDD et Gestion des Genres	11
Présentation et Objectif	11
4.2 Génération de Formulaires CRUD (Jeux Vidéo)	
Présentation et objectif.....	13
4.3 Gestion des Tournois (Relation ManyToOne)	15
Présentation et Objectif	15
4.4 Participants et Tournois (Relation ManyToMany).....	17
Présentation et Objectif	17
5. Conclusion et Perspectives	19
Prochaine étape : Mission 5 – Demande 5	19

DOSSIER AGORA MISSION 5.1 À 5.4

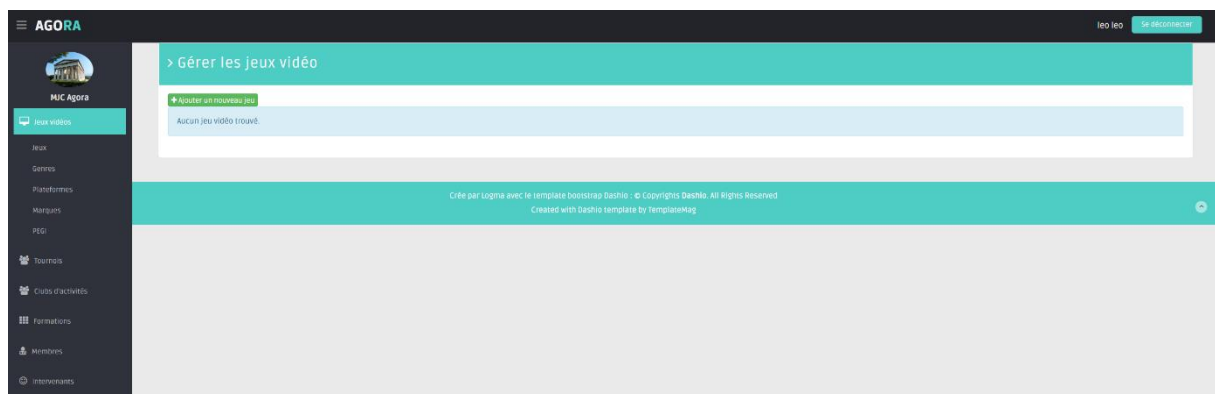
1. Contexte

Le projet AgoraBO est l'interface d'administration utilisée par la MJC Agora de Libreville pour gérer ses activités numériques (jeux, tournois, participants, classifications PEGI, plateformes, etc.). La MJC ne disposant pas de service informatique interne, l'ESN Logma accompagne la refonte progressive de cette application.

Les premières missions ont permis :

- De développer un back-office en PHP/MVC ;
- D'intégrer Twig comme moteur de templates pour séparer la logique et la présentation ;
- De migrer ce projet vers Symfony afin de bénéficier d'un routage structuré, d'une gestion des dépendances via Composer et d'une meilleure maintenabilité.

Le Sprint 5 s'inscrit dans cette continuité. Il introduit l'ORM Doctrine et vise à reconstruire les formulaires de gestion (CRUD) autour d'entités Doctrine plutôt que de requêtes SQL « manuelles ». L'objectif est de préparer une application plus robuste, plus sécurisée et plus facile à faire évoluer (notamment pour la future gestion des comptes et des droits utilisateurs).



Exemple vue générale de la section Jeux vidéo après la 5.4

DOSSIER AGORA MISSION 5.1 À 5.4

1.2 Composant Doctrine

Doctrine est le composant d'accès aux données utilisé par Symfony. C'est un ORM (Object Relational Mapper) qui fait le lien entre :

- Les classes PHP du projet (entités) ;
- Les tables de la base MySQL (schéma AgoraORM).

Dans le cadre d'AgoraBO, Doctrine est introduit pour :

- Remplacer la classe PDO « maison » par un accès objet centralisé ;
- Générer automatiquement les tables et leurs évolutions via les migrations ;
- Simplifier les opérations CRUD grâce aux repositories et à l'EntityManager ;
- Faciliter l'intégration avec les formulaires Symfony (FormType) et la sécurité.

Cette étape marque le passage d'un code fortement couplé à MySQL à un modèle orienté objet, plus proche du métier et plus simple à maintenir.

```
17  ###> symfony/framework-bundle ###
18  APP_ENV=dev
19  APP_SECRET=
20  ###< symfony/framework-bundle ###
21
22  ###> symfony/routing ###
23  # Configure how to generate URLs in non-HTTP contexts, such as CLI commands.
24  # See https://symfony.com/doc/current/routing.html#generating-urls-in-commands
25  DEFAULT_URI=http://localhost
26  ###< symfony/routing ###
27
28  #mes paramètres
29  AGORA_DSN="mysql:dbname=agora;host=localhost"
30  AGORA_DB_USER="adminLeo"
31  AGORA_DB_PWD="agora"
32
33  ###> doctrine/doctrine-bundle ###
34  # Format described at https://www.doctrine-project.org/projects/doctrine-dbal/en/latest/reference/configuration.html#connecting-using-a-url
35  # IMPORTANT: You MUST configure your server version, either here or in config/packages/doctrine.yaml
36  #
37  # DATABASE_URL="sqlite:///kernel.project_dir%/var/data_%kernel.environment%.db"
38  # DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=8.0.32&charset=utf8mb4"
39  # DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=10.11.2-MariaDB&charset=utf8mb4"
40  #DATABASE_URL="mysql://adminLeo:agora@127.0.0.1:3306/agoraorm?serverVersion=9.0.0"
41
42  # Symfony Doctrine DB config
43  #DATABASE_URL="mysql://adminLeo:agora@127.0.0.1:3306/agoraorm?serverVersion=10.4&charset=utf8mb4"
44
45  DATABASE_URL="mysql://adminLeo:agora@127.0.0.1:3306/agoraorm?serverVersion=9.0.0"
46  #DATABASE_URL="postgres://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=15&cha
47  # reset=utf8"
48  ###< doctrine/doctrine-bundle ###
```

Extrait du fichier .env montrant la variable DATABASE_URL

DOSSIER AGORA MISSION 5.1 À 5.4

1.3 Entités

Une entité Doctrine représente un objet métier stocké en base.

Elle est décrite par :

- Une classe PHP dans src/Entity ;
- Des attributs Doctrine ([ORM\Entity], [ORM\Column], etc.) qui indiquent le nom de la table et les colonnes ;
- Un repository associé qui regroupe les requêtes de lecture/écriture.

Dans AgoraBO, plusieurs entités ont été créées ou adaptées lors du Sprint 5 :

- Genre : identifiant et libellé du genre de jeu vidéo ;
- PEGI : niveaux de classification par âge ;
- Plateforme : type de console ou support ;
- Marque : éditeur ou marque associée au jeu ;
- Tournoi : tournoi organisé par la MJC (libellé, date, date de création, catégorie, etc.) ;
- CatTournois : catégories de tournois (ex : tournoi local, tournoi en ligne) ;
- Participant : personne inscrite à un ou plusieurs tournois.

Ces entités forment le socle du modèle de données Doctrine sur lequel reposent les formulaires CRUD du Sprint 5.

```
1  <?php
2
3  namespace App\Entity;
4
5  use App\Repository\GenreRepository;
6  use Doctrine\Common\Collections\ArrayCollection;
7  use Doctrine\Common\Collections\Collection;
8  use Doctrine\ORM\Mapping as ORM;
9
10 #[ORM\Entity(repositoryClass: GenreRepository::class)]
11 class Genre
12 {
13     #[ORM\Id]
14     #[ORM\GeneratedValue]
15     #[ORM\Column]
16     private ?int $id = null;
17
18     #[ORM\Column(name: 'lib_genre', length: 255)]
19     private ?string $libGenre = null;
20
21     /**
22      * @var Collection<int, JeuVideo>
23      */
24     #[ORM\OneToMany(targetEntity: JeuVideo::class, mappedBy: 'genre')]
```

Exemple : Initialisation de l'entité Genre.php

DOSSIER AGORA MISSION 5.1 À 5.4

1.4 Associations many to ones et many to many

Pour refléter correctement le métier de la MJC Agora, les entités sont reliées entre elles par différentes associations. Le Sprint 5 se concentre surtout sur deux types de relations Doctrine.

a) Associations many-to-one / one-to-many

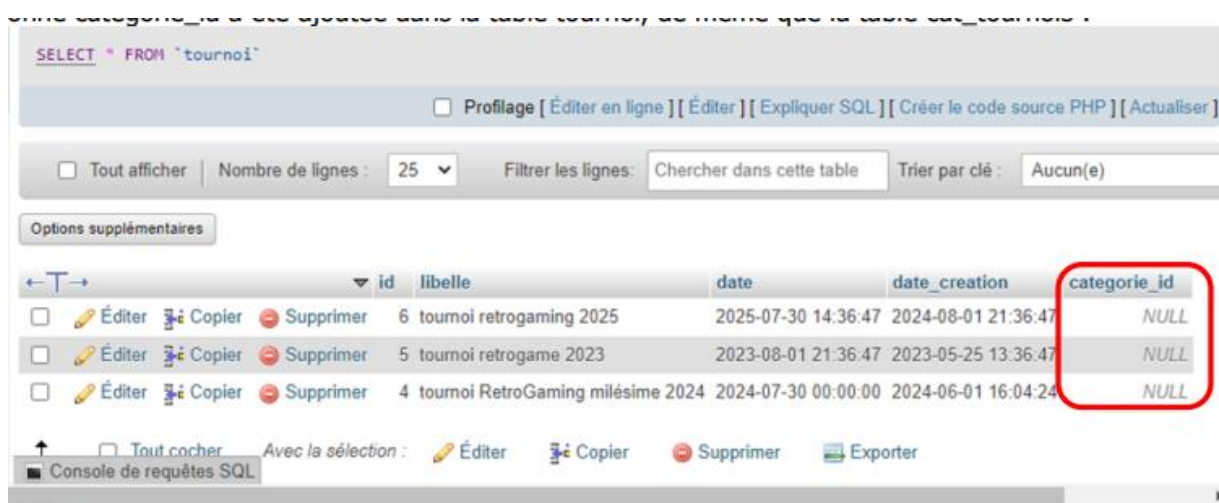
Une association many-to-one signifie que plusieurs enregistrements d'une entité sont liés à un seul enregistrement d'une autre entité. Dans AgoraBO :

- Chaque tournoi appartient à une seule catégorie de tournois (Tournoi → CatTournois) ;
- Une catégorie peut regrouper plusieurs tournois.

Doctrine représente cela par :

- Un attribut de type CatTournois dans l'entité Tournoi (many-to-one) ;
 - Une collection de tournois dans l'entité CatTournois (one-to-many).
- En base, cette relation se traduit par une clé étrangère `categorie_id` dans la table `tournoi`.

Une `categorie_id` a été ajoutée dans la table `tournoi`, de même que la table `cat_tournois`.



The screenshot shows the phpMyAdmin interface for the 'tournoi' table. The table structure is displayed with columns: id, libelle, date, date_creation, and categorie_id. The 'categorie_id' column is highlighted with a red box, indicating it is a foreign key. The table contains three rows of data, all with NULL values in the 'categorie_id' column.

	id	libelle	date	date_creation	categorie_id
☐	6	tournoi retrogaming 2025	2025-07-30 14:36:47	2024-08-01 21:36:47	NULL
☐	5	tournoi retrogame 2023	2023-08-01 21:36:47	2023-05-25 13:36:47	NULL
☐	4	tournoi RetroGaming milésime 2024	2024-07-30 00:00:00	2024-06-01 16:04:24	NULL



The screenshot shows the phpMyAdmin database structure view. The 'cat_tournois' table is highlighted with a red box. The table structure is as follows:

Table	Action	Lignes	Type	Int
cat_tournois	Parcourir Structure Rechercher Insérer Vider Supprimer	0	MyISAM	utf8
doctrine_migration_versions	Parcourir Structure Rechercher Insérer Vider Supprimer	2	MyISAM	utf8

Capture phpMyAdmin montrant la relation entre tournoi et cat_tournois

DOSSIER AGORA MISSION 5.1 À 5.4

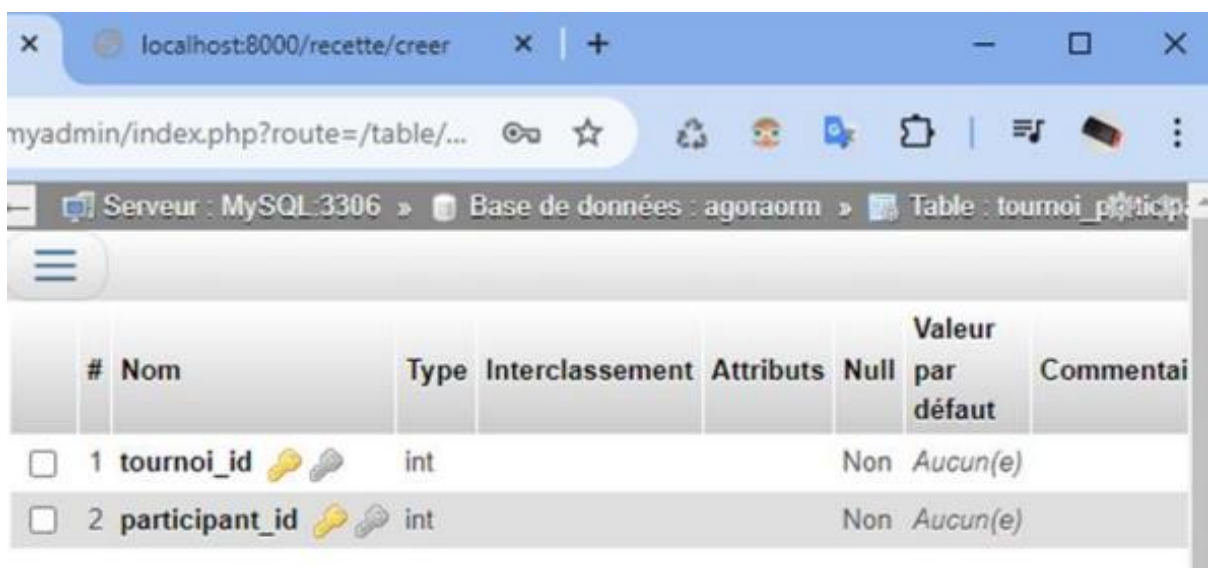
b) Associations many-to-many

Une association many-to-many indique qu'un enregistrement peut être lié à plusieurs éléments de l'autre entité, et inversement. Dans le projet :

- Un tournoi peut accueillir plusieurs participants ;
- Un participant peut être inscrit à plusieurs tournois.

Doctrine modélise cette relation en créant automatiquement une table de jointure (par exemple tournoi_participant) lors de l'exécution des migrations.

Les entités Tournoi et Participant contiennent chacune une collection représentant l'autre côté de la relation.



	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentai
☐	1	tournoi_id	int			Non	Aucun(e)	
☐	2	participant_id	int			Non	Aucun(e)	

Création de table tournoi_participant avec 2 clés étrangères (qui sont les clés primaires des tables tournoi et participant

DOSSIER AGORA MISSION 5.1 À 5.4

2. Présentation de l'équipe et organisation du groupe

L'équipe projet reste composée de quatre membres :

- Léo
- Kamdine
- Ryan
- Quentin

Le fonctionnement du groupe repose sur les principes suivants :

- Léo joue le rôle de chef de projet. Il s'occupe de la branche principale sur GitHub et de la coordination générale. Il est également fortement impliqué sur la partie tournois (relations many-to-one et many-to-many).
- Kamdine prend en charge la génération des formulaires CRUD avec Doctrine pour Genre, puis l'adaptation du même principe à d'autres entités (plateformes notamment) ainsi que de la rédaction du compte rendu hebdomadaire.
- Ryan se concentre sur la gestion des PEGI (entité et CRUD associés) et participe aux tests fonctionnels et la création des cartes dans Trello.
- Quentin travaille sur la gestion des plateformes et sur l'harmonisation des vues Twig afin que les CRUD générés respectent l'interface AgoraBO.

Les décisions techniques importantes (nommage des entités, choix des relations, structure du menu) sont discutées en commun. Le code est relu avant intégration afin de garder une base cohérente et de limiter les régressions.

Différents outils ont été mis à disposition pour le bon déroulement du Sprint 5 :



DOSSIER AGORA MISSION 5.1 À 5.4

3. Organisation du travail

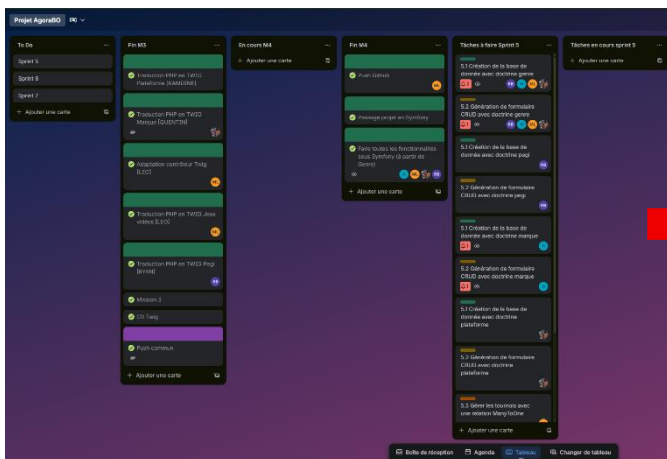
3.1 Trello

Le tableau Trello « Projet AgoraBO » sert de support principal au suivi des sprints. On y distingue plusieurs colonnes :

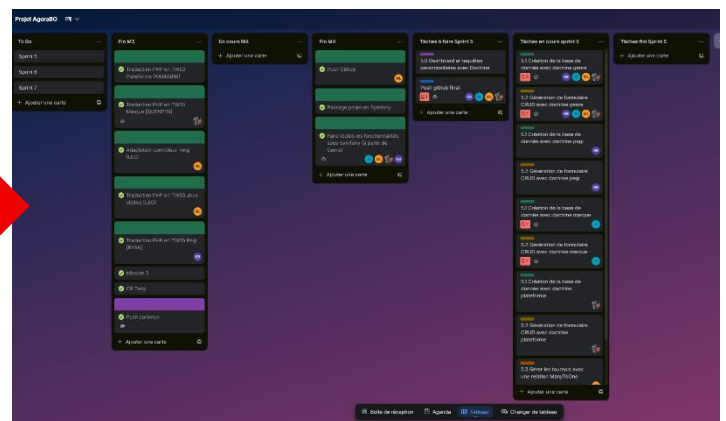
- « To Do » pour les sprints planifiés (Sprint 5, Sprint 6, Sprint 7) ;
- « Fin M3 » et « Fin M4 » pour les missions déjà terminées (Twig, passage complet sous Symfony) ;
- « Tâches à faire Sprint 5 » avec les cartes détaillées pour chaque demande (5.1 à 5.5) ;
- « Tâches en cours Sprint 5 » puis « Tâches fini Sprint 5 » pour suivre l'avancement réel des tâches Doctrine.

Chaque carte est associée à un ou plusieurs membres, ce qui permet d'identifier rapidement qui est responsable de chaque partie (création de la base, CRUD genre, CRUD PEGI, CRUD plateformes, tournois, etc.).

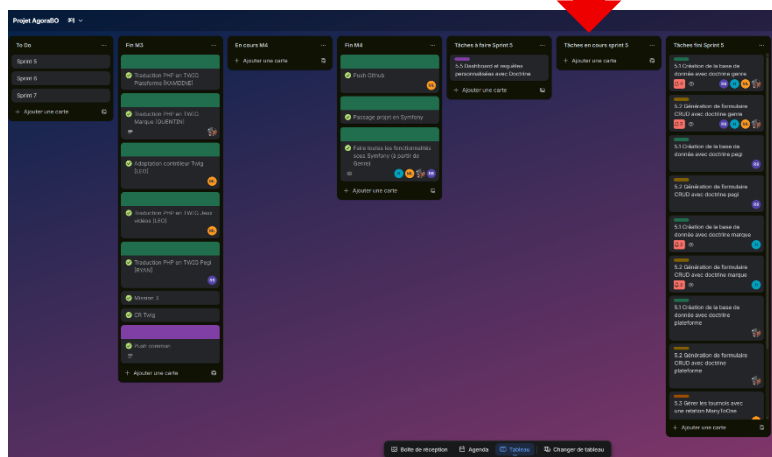
Processus Trello AVANT-PENDANT-APRES les 4 demandes du Sprint 5 :



AVANT



PENDANT



APRES

DOSSIER AGORA MISSION 5.1 À 5.4

3.2 Github

Le code du projet est versionné sur GitHub, dans le dépôt :

<https://github.com/Lavarice/Mission4.git>

La branche dédiée au sprint Doctrine regroupe tous les commits liés :

- À la création des entités ;
- Aux migrations ;
- Aux contrôleurs CRUD ;
- Aux formulaires et vues Twig associés.

Léo et Kamdine se charge de la gestion des merges. Les bonnes pratiques suivantes sont appliquées :

- Git pull systématique avant de commencer une nouvelle tâche ;
- Commits fréquents avec messages explicites ;
- Push sur la branche sprint, puis validation avant intégration.

Les tâches sont réparties entre les membres et plus particulièrement pour l'adaptation des CRUDS afin d'optimiser le temps consacré à la tâche

This branch is 19 commits ahead of main.

Contribute

Kamdinee Ajout du tournoi du participant fcd023a · 8 hours ago 24 Commits

File/Folder	Commit Message	Time
bin	quetin 5.1	last week
config	quetin 5.1	last week
migrations	5.3 jeuvideo fonctionnel	last week
public	quetin 5.1	last week
src	Ajout du tournoi du participant	8 hours ago
templates	Amélioration du CRUD Participants : liste des tournois et me...	8 hours ago
.editorconfig	quetin 5.1	last week
.env	5.3 jeuvideo fonctionnel	last week
.env.dev	quetin 5.1	last week
.gitignore	quetin 5.1	last week
composer.json	quetin 5.1	last week
composer.lock	quetin 5.1	last week
sprint 5 Agora demande 3 - gérer les tournois ...	fix	9 hours ago

Projet SIO2
github.com/Lavarice/Mission4.git

symfony

Activity
0 stars
0 watching
0 forks
Report repository

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Contributors 2
MartinLe0
Kamdinee Kamdine

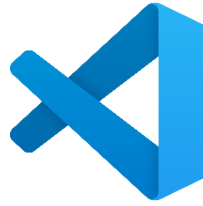
Page d'accueil du dépôt GitHub Mission4 avec les différents dossiers/commits

DOSSIER AGORA MISSION 5.1 À 5.4

3.3 Outils utilisés

Pour ce sprint, les principaux outils sont :

- Symfony CLI pour la génération des entités, migrations et CRUD ;
- Composer pour l'installation des composants nécessaires (form, validator, Twig, security-csrf) ;
- Visual Studio Code comme environnement de développement ;
- WampServer / MySQL + phpMyAdmin pour l'hébergement et l'inspection de la base AgoraORM.



DOSSIER AGORA MISSION 5.1 À 5.4

4. Développement et processus du Sprint

4.1 Mission 1 : Création de la BDD et Gestion des Genres

Présentation et Objectif

L'objectif de cette première étape était de mettre en place les fondations de l'application en créant la base de données et en gérant une première entité simple. Il s'agissait de définir la structure de données pour les "Genres" de jeux vidéo (ex: Action, Aventure, RPG) et de permettre à l'administrateur de les gérer (ajouter, modifier, supprimer) via une interface web, en utilisant l'ORM Doctrine pour l'abstraction de la base de données.

```
1  <?php
2
3  namespace App\Entity;
4
5  use App\Repository\GenreRepository;
6  use Doctrine\Common\Collections\ArrayCollection;
7  use Doctrine\Common\Collections\Collection;
8  use Doctrine\ORM\Mapping as ORM;
9
10 #[ORM\Entity(repositoryClass: GenreRepository::class)]
11 class Genre
12 {
13     #[ORM\Id]
14     #[ORM\GeneratedValue]
15     #[ORM\Column]
16     private ?int $id = null;
17
18     #[ORM\Column(name: 'lib_genre', length: 255)]
19     private ?string $libGenre = null;
20
21     /**
22      * @var Collection<int, JeuVideo>
23      */
24     #[ORM\OneToOne(targetEntity: JeuVideo::class, mappedBy: 'genre')]
```

Annotation ORM dans le fichier src/Entity/Genre.php

```
25     #[Route('/new', name: 'app_genre_new', methods: ['GET', 'POST'])]
26     public function new(Request $request, EntityManagerInterface $entityManager): Response
27     {
28         $genre = new Genre();
29         $form = $this->createForm(GenreType::class, $genre);
30         $form->handleRequest($request);
31
32         if ($form->isSubmitted() && $form->isValid()) {
33             $entityManager->persist($genre);
34             $entityManager->flush();
35
36             return $this->redirectToRoute('app_genre_index', [], Response::HTTP_SEE_OTHER);
37         }
38
39         return $this->render('genre/new.html.twig', [
40             'genre' => $genre,
41             'form' => $form,
42             'menuActif' => 'Jeux',
43         ]);
44     }
45 }
```

Méthodes New dans le fichier src/Controller/GenreController.php

DOSSIER AGORA MISSION 5.1 À 5.4

Identif	Libellé	Jeux	
1	Action	Aucun jeu	
2	Aventure	Aucun jeu	
3	RPG	Aucun jeu	
4	Simulation	Aucun jeu	
5	Sport	Aucun jeu	

Crée par Logma avec le template bootstrap Dashio : © Copyrights Dashio. All Rights Reserved
Created with Dashio template by TemplateMag

Capture d'écran de la liste des genres de l'application

Explication

L'emploi de l'ORM Doctrine constitue un choix stratégique pour la pérennité du projet. En définissant nos données via des entités PHP, nous nous affranchissons des spécificités du langage SQL, ce qui rend le code plus maintenable et évolutif. Le mécanisme de migrations offre l'avantage majeur de versionner la structure de la base de données, permettant un déploiement fiable et reproductible des tables, comme celle des Genres, sans intervention manuelle risquée.

DOSSIER AGORA MISSION 5.1 À 5.4

4.2 Génération de Formulaires CRUD (Jeux Vidéo)

Présentation et objectif

Cette mission visait à gérer l'entité centrale de l'application : les Jeux Vidéo. L'objectif était de créer un système complet pour ajouter et éditer des jeux, ce qui implique des formulaires plus complexes. En effet, un jeu vidéo est lié à plusieurs autres données (Genre, Plateforme, PEGI, Marque), nécessitant des listes déroulantes dynamiques et des champs spécifiques (dates, prix) pour garantir l'intégrité des données saisies.

```
class JeuVideoType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('refJeu', TextType::class, [
                'label' => 'Référence du jeu',
                'attr' => ['class' => 'form-control'],
            ])
            ->add('nom', TextType::class, [
                'label' => 'Nom du jeu',
                'attr' => ['class' => 'form-control'],
            ])
            ->add('prix', MoneyType::class, [
                'label' => 'Prix',
                'currency' => 'EUR',
                'attr' => ['class' => 'form-control'],
            ])
            ->add('dateParution', DateType::class, [
                'label' => 'Date de parution',
                'widget' => 'single_text',
                'attr' => ['class' => 'form-control'],
            ])
            ->add('genre', EntityType::class, [
                'class' => Genre::class,
                'choice_label' => 'libGenre',
                'label' => 'Genre',
                'attr' => ['class' => 'form-control'],
            ])
    }
}
```

Méthode buildForm avec les différents types de champs dans le fichier
src/Form/JeuVideoType.php

> Créer un nouveau jeu vidéo

Référence du jeu

Nom du jeu

Prix

Date de parution

Genre

Action

Classification PEGI

Plateforme

Marque

Enregistrer Annuler

Retour à la liste

Crée par Logma avec le template bootstrap Dashio : © Copyrights Dashio. All Rights Reserved.
Created with Dashio template by TemplateMag

Capture d'écran du formulaire d'ajout d'un jeu vidéo

DOSSIER AGORA MISSION 5.1 À 5.4

The screenshot shows the 'Éditer le jeu vidéo: Zelda' form in the AGORA application. The form is titled 'Éditer le jeu vidéo: Zelda' and contains the following fields:

- Titre: Input field with 'Zelda' entered.
- Genre: Dropdown menu with 'Aventure' selected.
- Classification PEGI: Dropdown menu with '7+' selected.
- Plateforme: Dropdown menu with 'PC' selected.
- Marque: Dropdown menu with 'Sony' selected.
- Statut: Radio buttons for 'Actif' (selected) and 'Archivé'.
- Actions: Buttons for 'Ajouter à la liste', 'Supprimer', and 'Retour à la liste'.

The form is part of a larger application interface with a sidebar menu on the left and a top navigation bar. The sidebar menu includes links for 'Jeux vidéo', 'Jeux', 'Genres', 'Plateformes', 'Marques', 'PGL', 'Tournois', 'Clubs d'activités', 'Formations', 'Membres', and 'Membres'. The top navigation bar shows the user 'leo leo' and a 'Se déconnecter' button.

Capture d'écran du formulaire d'édition d'un jeu vidéo (Zelda)

Explication

Pour garantir la fiabilité de la saisie des données, nous avons opté pour le composant Form de Symfony. Cette approche centralisée permet de définir la structure, les types de champs et les règles de validation au sein d'une classe dédiée (JeuVideoType).

L'avantage principal réside dans l'automatisation du rendu HTML et du traitement des soumissions, assurant ainsi une sécurité accrue contre les failles courantes (CSRF, injection) tout en offrant une interface utilisateur cohérente pour la gestion des jeux.

DOSSIER AGORA MISSION 5.1 À 5.4

4.3 Gestion des Tournois (Relation ManyToOne)

Présentation et Objectif

Pour la gestion des tournois, l'objectif était d'implémenter et de gérer des relations de type "Many-To-One" (Plusieurs-à-Un). Un tournoi doit être obligatoirement associé à une plateforme spécifique et à une catégorie de tournoi. L'interface devait permettre de sélectionner ces associations lors de la création d'un tournoi, assurant ainsi que chaque événement est correctement catégorisé.

```
#[ORM\ManyToOne(targetEntity: Plateformes::class)]
#[ORM\JoinColumn(nullable: false)]
private ?Plateformes $plateforme = null;
```

```
public function setPlateforme(?Plateformes $plateforme): static
{
    $this->plateforme = $plateforme;

    return $this;
}
```

Propriété \$plateforme avec l'attribut ManyToOne dans le fichier src/Entity/Tournoi.php

```
)
->add('plateforme', EntityType::class, [
    'class' => Plateformes::class,
    'choice_label' => 'libPlateforme',
])
```

Champ ajout plateforme dans le fichier src/Form/TournoiType.php

> Gérer les tournois					
Id	Libelle	Date	Date de creation	Participants	
1	Mario Kart	2025-12-04	2025-11-01	kam	 
2	Minecraft	2025-12-05	2025-11-01	AUCUN	 
					

Capture d'écran du site Agora dans la section Gérer les tournois

DOSSIER AGORA MISSION 5.1 À 5.4

> Modifier un tournoi

Nom

Date debut

Date fin

Nb participants

Description

Plateforme

Catégorie

Participants

[Retour à la liste](#)

Modification d'un tournoi sur le site avec la liste déroulante de la plateforme

> Gérer les catégories de tournois

Id	Libelle	
1	Tournoi Local	
2	Tournoi Régional	
3	Tournoi National	

Gérer les différentes catégories de tournoi avec 3 libellés : Tournoi local, régional et national

Explication

L'intégration de la relation ManyToOne via le formulaire TournoiType répond à un double impératif : ergonomie et intégrité des données. En proposant une sélection guidée des plateformes via un composant EntityType, nous éliminons les risques d'erreurs de saisie. Techniquement, Doctrine gère la complexité des clés étrangères de manière transparente, garantissant que chaque tournoi est systématiquement et correctement rattaché à une plateforme existante.

DOSSIER AGORA MISSION 5.1 À 5.4

4.4 Participants et Tournois (Relation ManyToMany)

Présentation et Objectif

La dernière mission, la plus complexe, consistait à gérer les inscriptions des participants aux tournois. Il s'agit d'une relation "Many-To-Many" (Plusieurs-à-Plusieurs) : un participant peut s'inscrire à plusieurs tournois, et un tournoi accueille plusieurs participants. L'objectif était de permettre cette gestion directement depuis la fiche d'un participant, en sélectionnant plusieurs tournois via une liste, tout en garantissant que ces modifications soient correctement sauvegardées.

```
1  <?php
2  |
3  namespace App\Form;
4
5  use App\Entity\Participant;
6  use App\Entity\Tournoi;
7  use Symfony\Bridge\Doctrine\Form\Type\EntityType;
8  use Symfony\Component\Form\AbstractType;
9  use Symfony\Component\Form\FormBuilderInterface;
10 use Symfony\Component\OptionsResolver\OptionsResolver;
11
12 class ParticipantType extends AbstractType
13 {
14     public function buildForm(FormBuilderInterface $builder, array $options): void
15     {
16         $builder
17             ->add('prenom')
18             ->add('nom')
19             ->add('telephone')
20             ->add('email')
21             ->add('tournois', EntityType::class, [
22                 'class' => Tournoi::class,
23                 'choice_label' => 'nom',
24                 'multiple' => true,
25                 'by_reference' => false,
26             ])
27     ;
28 }
```

Champ tournoi avec l'option 'by_reference' => false dans le fichier
src/Form/ParticipantType.php

DOSSIER AGORA MISSION 5.1 À 5.4

AGORA

MJC Agora

Jeux vidéos

Tournois

Tournois

Catégorie de tournois

Participants

Clubs d'activités

Formations

Edit Participant

Prenom: kam

Nom: yo

Telephone: 0624453232

Email: aaa@aa.fr

Tournois: Mario Kart Minecraft

Update

[back to list](#)

Delete

Capture d'écran de l'édition d'un participant montrant la sélection multiple des tournois

AGORA

MJC Agora

Jeux vidéos

Tournois

Tournois

Catégorie de tournois

Participants

> Gérer les tournois

Id	Libelle	Date	Date de creation	Participants
1	Mario Kart	2025-12-04	2025-11-01	kam
2	Minecraft	2025-12-05	2025-11-01	leo

Capture d'écran pour gérer les tournois avec l'affichage des participants (plusieurs par tournoi sont possibles)

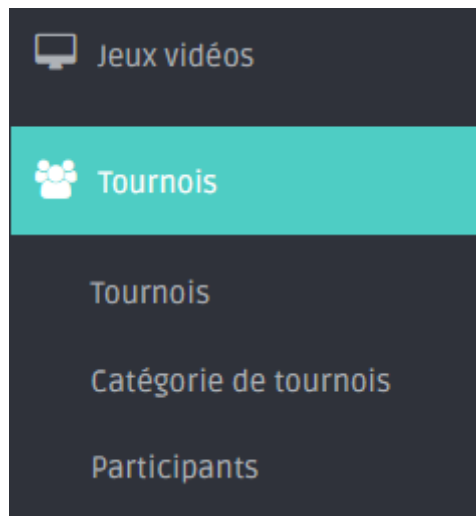
Explication

La gestion de la relation ManyToMany entre participants et tournois représente un défi technique intéressant en termes d'expérience utilisateur et de persistance. L'objectif était de simplifier l'inscription multiple sans compromettre la stabilité des données. La configuration spécifique du formulaire (option 'by_reference' => false) permet de déléguer la logique d'association aux méthodes de l'entité. Cette méthode assure une synchronisation parfaite de la table de jointure, offrant ainsi une gestion fluide et robuste des inscriptions directement depuis la fiche participante.

DOSSIER AGORA MISSION 5.1 À 5.4

5. Conclusion et Perspectives

L'ensemble de ces 4 missions a permis de construire le socle technique robuste de l'application Agora. Nous avons réussi à modéliser et à manipuler des données complexes, allant de simples entités (Genres) à des relations multiples et croisées (Participants/Tournois). Cette base solide garantit non seulement l'intégrité des données de l'association, mais offre également une interface d'administration fonctionnelle et intuitive pour gérer l'ensemble du catalogue de jeux et des événements e-sport.



Prochaine étape : Mission 5 – Demande 5

La prochaine étape du développement se concentrera sur l'analyse et la visualisation de ces données. Le directeur de l'association a exprimé le besoin d'un Tableau de Bord (Dashboard) pour piloter l'activité.